

+

New

AI Chat

Home

Skills

Clear

Workflows

Drive

...

More

Engine_1.py architectural and stability review request 2

Engine_1.py

65.19 KB

I am providing a Python script named 'Engine_1.py'. It is a production-grade cryptocurrency algorithmic trading hub that combines live Binance footprint data streaming (via websockets) with Playwright-based browser scraping (via Coinglass) for macro indicators. It aggregates this data and feeds it into pre-trained LightGBM models every 15-minute candle to generate trading signals, managing up to 4 concurrent trades with strict risk management parameters (ATR-based SL/TP).

We have just completed a refactoring pass to remove bare 'except' blocks, enforce atomic file writes for its historical caching ('combined_seed_history.xlsx'), and optimize dynamic module imports from its hot prediction loops.

I need you to perform a ruthless, expert-level architectural and stability review of this code. Do not focus on basic PEP8 formatting or cosmetic nits. Instead, analyze the code across the following dimensions:

1. Asynchronous Stability & Concurrency: Are there any hidden race conditions, deadlocks, or memory leak risks in the 'asyncio.gather' tasks, 'asyncio.Lock' usage, or Playwright browser lifecycle management?
2. Fault Tolerance: How resilient is the WebSocket feed and the UI-scraping loops? Are there any unhandled edge cases where network drops could cause the state machine to desync or hang indefinitely?
3. Trading Execution Logic: Review 'Engine1TradeTracker' and the SL/TP logic. Are there any floating-point edge cases or logical flaws that could result in improper exit triggers or incorrect position sizing?
4. Resource Exhaustion: Does the continuous appending to 'collections.deque' or the periodic pandas/numpy conversions risk memory ballooning over a multi-week continuous runtime?

Please provide a highly concise, prioritized list of any critical vulnerabilities or structural flaws you find, followed by concrete, minimal code changes to fix them.

Below is the ruthless version. I'd fix the first 4 items before allowing this to trade unattended.

Executive verdict

The biggest risks are not deque growth or pandas memory. The critical risks are:

1. Bad/zero scraped prices can falsely close trades at catastrophic prices.
2. Your "closed candle" prediction logic can fire mid-candle after seeding/restart.
3. Heavy pandas/LightGBM work runs inside the asyncio event loop.
4. Watchdog heartbeats are based on loop attempts, not successful data extraction.
5. The Binance "footprint" feed is REST kline polling, not WebSocket/tick footprint.

P0 — Critical structural flaws

1. Invalid Coinglass DOM values can trigger false SL/TP exits

parse_float("N/A") -> 0.0. Then SnapshotStore.update() applies price=0.0, and immediately:

```
self.trade_tracker.check_exits(symbol, new_snap.price)
```

A single bad DOM read can close a long at 0.0 or a short at an absurd TP.

Minimal fix

Add finite/valid price guards and only check exits when the patch contains a valid fresh price.

```
import math

def finite_float_or_none(v: Any) -> Optional[float]:
    try:
        x = float(v)
    except (TypeError, ValueError):
        return None
    return x if math.isfinite(x) else None

def valid_price(v: Any) -> bool:
    x = finite_float_or_none(v)
    return x is not None and x > 0.0
```

Replace SnapshotStore.update() with this safer version:

```
async def update(self, symbol: str, **patch: Any) -> None:
    if symbol not in self._data:
        return

    async with self._locks[symbol]:
        cur = self._data[symbol]

        clean_patch = {}
        for k, v in patch.items():
            if not hasattr(cur, k):
                continue

            if k in _FLOAT_FIELDS:
                fv = finite_float_or_none(v)
                if fv is None:
                    continue

                # Never allow invalid/zero price to overwrite good state.
                if k == "price" and fv <= 0.0:
                    continue

                clean_patch[k] = fv
            else:
                clean_patch[k] = v

        if not clean_patch:
            return

        self._seq += 1
        new_snap = dataclasses.replace(
            cur,
            seq=self._seq,
            ts_ns=time.time_ns(),
            **clean_patch
        )

        price_updated = "price" in clean_patch

        if self.trade_tracker and price_updated:
            self.trade_tracker.check_exits(symbol, new_snap.price)
            self.trade_tracker.update_live_pnl(symbol, new_snap.price)

        if self.predictor and new_snap.price > 0.0:
            new_snap = await asyncio.to_thread(
                self.predictor.on_tick_update,
                symbol,
                new_snap,
                self.trade_tracker
            )
```

```
self_data[symbol] = new_snap
```

[Copy](#)

Also update Binance feed to provide a reliable crypto price:

```
await self.store.update(
    sym,
    price=close_price,
    fp_delta=candle.delta,
    fp_poc=candle.poc
)
```

[Copy](#)

This makes trade exits less dependent on Coinglass DOM freshness.

2. Candle-close prediction can fire mid-candle after seeding/restart

Your seed history includes the current 15m candle:

```
computed_timestamps = get_historical_timestamps(symbol, current_time, target_steps)
```

[Copy](#)

The first timestamp is the current candle open. Then live predictor sees history[-1] and may predict/enter immediately, even though that candle is not closed.

Also, on the next rollover, this block can append a duplicate candle:

```
if symbol in self.current_candle:
    history.append(self.current_candle[symbol])
```

[Copy](#)

Minimal fix

Only keep closed candles in history and initialize `_last_predict_bar` so restart does not fire stale entries.

Replace `set_history()`:

```
def set_history(self, symbol: str, candles: collections.deque | list) -> None:
    now_open = int(time.time() // 900) * 900

    cleaned = []
    for c in candles:
        try:
            ot = int(c.get("open_time", 0))
        except Exception:
            continue

        # Keep only closed bars.
        if ot > 0 and ot < now_open:
            row = dict(c)
            row["open_time"] = ot
            cleaned.append(row)

    cleaned.sort(key=lambda r: r["open_time"])
    cleaned = cleaned[-1200:]

    self.candles_history[symbol] = collections.deque(cleaned, maxlen=1200)

    # Prevent immediate stale/mid-candle prediction after seed/restart.
    if cleaned:
        self._last_predict_bar[symbol] = cleaned[-1]["open_time"]
```

[Copy](#)

In `load_history_from_disk()`, replace:

```
self.candles_history[sym] = collections.deque(candle_list, maxlen=1200)
```

[Copy](#)

with:

```
self.set_history(sym, candle_list)
```

[Copy](#)

Then in `on_tick_update()`, change the rollover append to avoid duplicates:

```
if symbol not in self.current_candle or self.current_candle[symbol].get("open_time") != open_time:
    prev = self.current_candle.get(symbol)

    if prev and int(prev.get("open_time", 0)) < open_time:
        prev_ot = int(prev["open_time"])
        if not history or int(history[-1].get("open_time", 0)) != prev_ot:
            history.append(dict(prev))

    self.current_candle[symbol] = {
        "open_time": open_time,
        "open": snap.price,
        "high": snap.price,
        "low": snap.price,
        "close": snap.price,
        "volume": snap.volume,
        "rsi": snap.rsi,
        "fut_cvd": snap.fut_cvd,
        "spot_cvd": snap.spot_cvd,
        "funding": snap.funding,
        "liq_long": snap.liq_long,
        "liq_short": snap.liq_short,
        "ls_ratio": snap.ls_ratio,
        "oi": snap.oi,
        "coins_bid": snap.coins_bid,
        "coins_ask": snap.coins_ask,
        "dollars_bid": snap.dollars_bid,
        "dollars_ask": snap.dollars_ask,
        "whale_idx": snap.whale_idx,
        "tk_buy_cnt": snap.tk_buy_cnt,
        "tk_sell_cnt": snap.tk_sell_cnt,
        "fp_poc": snap.fp_poc,
    }
```

[Copy](#)

3. LightGBM/pandas prediction blocks the event loop

`predictor.on_tick_update()` performs pandas transforms and LightGBM inference inside `SnapshotStore.update()`, which is called by Playwright/Binance feed loops.

At a 15m boundary, many symbols can trigger feature generation nearly simultaneously. This can freeze:

- Playwright scraping
- Binance polling
- renderer
- watchdog

Minimal fix

You already saw the `asyncio.to_thread()` change above. Add a predictor-level lock because predictor state is shared across symbols:

```
class LiveStrategyPredictor:
    def __init__(self, symbols: List[str]):
        self._lock = threading.RLock()
        ...
```

Then wrap the body of `on_tick_update()`:

```
def on_tick_update(self, symbol: str, snap: AssetSnapshot, trade_tracker: Any = None) -> AssetSnapshot:
    with self._lock:
        return self._on_tick_update(symbol, snap, trade_tracker)

def _on_tick_update(self, symbol: str, snap: AssetSnapshot, trade_tracker: Any = None) -> AssetSnapshot:
    # move the existing on_tick_update body here
    ...
```

Without this, moving prediction to a thread can introduce real shared-state races around `candles_history`, `current_candle`, and BTC reference history.

4. Watchdog heartbeat is misleading

Current loops do this:

```
self.last_heartbeat_ns = time.time_ns()
await asyncio.gather(...)
```

That means the system looks healthy even if every frame extraction fails quickly.

Also:

```
self.page.on("response", lambda r: asyncio.create_task(handle_response(r)))
```

creates untracked tasks forever. During heavy network activity or broken pages, this can become a leak.

Minimal fix

Track response tasks:

```
class CoinglassTab:
    def __init__(...):
        ...
        self._response_tasks: set[asyncio.Task] = set()
        self._poll_failures = 0
```

In `start()`:

```
def _spawn_response_task(response):
    task = asyncio.create_task(handle_response(response))
    self._response_tasks.add(task)
    task.add_done_callback(self._response_tasks.discard)

self.page.on("response", _spawn_response_task)
```

Make `_fetch_frame()` return `True` only after a successful store update. Also reject symbol mismatches:

```
if sym_actual and sym_actual != sym:
    print(f"[{self.tab_id}] [WARN] Window {win_idx} expected {sym}, got {sym_actual}; skipping.")
    return False
```

Then in `poll_loop()`:

```
results = await asyncio.gather(
    *[_fetch_frame(i) for i in range(1, 10)],
    return_exceptions=True
)

ok = sum(r is True for r in results)

if ok:
    self.last_heartbeat_ns = time.time_ns()
    self._poll_failures = 0
else:
    self._poll_failures += 1
```

Update watchdog to also recover dead poll tasks:

```
task_dead = (
    isinstance(c, CoinglassTab)
    and c in tab_tasks
    and tab_tasks[c].done()
)

stale = (
    hasattr(c, "last_heartbeat_ns")
    and now - c.last_heartbeat_ns > 20_000_000_000
)

if task_dead or stale:
    ...
```

P1 — High-priority trading/runtime issues

5. Trade sizing needs hard guards

Current sizing:

```
units = (self.current_capital * 0.005) / abs(entry_price - sl)
```

Problems:

- negative `current_capital` can create negative units
- tiny ATR/SL distance can create enormous units
- invalid `sl/tp/entry_price` can poison state
- no directional sanity check

Minimal fix

Inside `trigger_entry()`:

```
def trigger_entry(
    self,
    symbol: str,
```

```

        direction: int,
        entry_price: float,
        sl: float,
        tp: float,
        atr: float,
        macro: int,
        vol_regime: float
    ) -> None:
        with self.lock:
            if symbol in self.active_trades:
                return
            if len(self.active_trades) >= 4:
                return
            if direction not in (1, -1):
                return
            if not all(valid_price(x) for x in (entry_price, sl, tp, atr)):
                return

            if direction == 1 and not (sl < entry_price < tp):
                return
            if direction == -1 and not (tp < entry_price < sl):
                return

            stop_dist = abs(entry_price - sl)
            min_stop = max(
                globals().get("TICK_SIZES", {}).get(symbol, 0.0),
                entry_price * 1e-8,
                1e-12,
            )

            risk_capital = max(0.0, self.current_capital) * 0.005

            if risk_capital <= 0.0 or stop_dist <= min_stop:
                return

            units = risk_capital / stop_dist

            trade_id = f"{symbol}_{'LONG' if direction == 1 else 'SHORT'}_{int(time.time())}"

            self.active_trades[symbol] = {
                ...
                "units": units,
                ...
            }

        self.save_history()

```

Also guard live updates:

```

def update_live_pnl(self, symbol: str, current_price: float) -> None:
    if not valid_price(current_price):
        return
    ...

```

```

def check_exits(self, symbol: str, current_price: float) -> None:
    if not valid_price(current_price):
        return
    ...

```

And do not silently swallow persistence failures:

```

except Exception as e:
    print(f"[TradeTracker] [ERROR] Failed to save history: {e}")

```

6. Seeding can hang startup forever

The watchdog is only started after seeding. If one Playwright operation hangs while holding `focus_lock`, the whole startup can stall.

Minimal fix

Wrap per-symbol seeding with timeout and isolate failures:

```

async def seed_wrapper(tab: CoinglassTab, sym: str):
    try:
        await asyncio.wait_for(
            tab.seed_symbol(sym, excel_pool, focus_lock),
            timeout=15 * 60,
        )
    except Exception as e:
        print(f"[Setup] [WARN] Seeding failed for {sym}: {e}")

await asyncio.gather(*seeding_tasks, return_exceptions=True)

```

This prevents one bad chart/window from killing the whole engine.

7. Path handling is inconsistent

Some paths use:

```

os.path.dirname(os.path.abspath(__file__))

```

Others use:

```

"Engine_1/Seeding/..."
os.getcwd()

```

If the script is launched from a different working directory, seeding, screenshots, model retraining, and combined cache loading can silently diverge.

Minimal fix

At top:

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
SEED_DIR = os.path.join(BASE_DIR, "Seeding")
ALPHA_DIR = os.path.join(BASE_DIR, "alpha_squeezer_v17")
MODEL_DIR = os.path.join(ALPHA_DIR, "models")
CONFIG_DIR = os.path.join(ALPHA_DIR, "agent5_configs")

os.makedirs(SEED_DIR, exist_ok=True)

```

Then replace relative paths like:

```

"Engine_1/Seeding/snapshot_debug.json"

```

with:



```
os.path.join(SEED_DIR, "snapshot_debug.json")
```

[Copy](#)

For combined workbook atomic writes, prefer a real .xlsx temp file in the same directory:

```
combined_filename = os.path.join(SEED_DIR, "combined_seed_history.xlsx")

fd, tmp_filename = tempfile.mkstemp(
    prefix="combined_seed_",
    suffix=".xlsx",
    dir=SEED_DIR,
)
os.close(fd)

try:
    combined_wb.save(tmp_filename)
    os.replace(tmp_filename, combined_filename)
finally:
    if os.path.exists(tmp_filename):
        try:
            os.remove(tmp_filename)
        except OSError:
            pass
```

[Copy](#)

8. Hardcoded Coinglass credentials are a serious operational risk

This is not cosmetic. The pasted script contains a live-looking email/password fallback.

Replace:



```
email = os.environ.get("COINGLASS_EMAIL", "...")
password = os.environ.get("COINGLASS_PASSWORD", "...")
```

[Copy](#)

with:

```
email = os.environ.get("COINGLASS_EMAIL")
password = os.environ.get("COINGLASS_PASSWORD")

if not email or not password:
    raise RuntimeError("Set COINGLASS_EMAIL and COINGLASS_PASSWORD environment variables.")
```

[Copy](#)

Also rotate the exposed password.

P2 — Resource exhaustion / long-runtime issues

9. Deques are mostly safe; response tasks and raw legend extraction are not

The bounded deques are fine:

- `candles_history`: maxlen=1200
- `current_candle`: one per symbol
- `FootprintCandle.volume_profile`: cleared per candle

The real runtime pressure points are:

1. untracked Playwright response tasks
2. `rawLegends` returned on every live poll
3. pandas/LGBM blocking causing task backlog
4. unbounded trade history JSON over months



Minimal raw legend fix

Change JS function signature:

```
(includeRaw=false) => {
```

[Copy](#)

Then replace final return:

```
let out = { success: true, data: data };
if (includeRaw) {
    out.rawLegends = legends.map(e1 => e1.innerText || '');
}
return out;
```

[Copy](#)

Live polling:

```
res = await frame.evaluate(SINGLE_FRAME_EXTRACTION_JS, False)
```

[Copy](#)

BTC seeding debug only:

```
res = await frame.evaluate(SINGLE_FRAME_EXTRACTION_JS, symbol == "BTCUSD1")
```

[Copy](#)

This reduces browser-to-Python payload size significantly.

10. Binance “footprint” is not actually WebSocket/tick footprint

This file imports websockets, but `BinanceFootprintfeed` polls:

```
https://fapi.binance.com/fapi/v1/klines
```

[Copy](#)

every 0.5s.

Also, this logic:

```
self.volume_profile[bucket] += buy_vol + sell_vol
```

[Copy](#)

does not build a real volume-at-price profile. It only assigns cumulative kline volume to the latest close-price bucket.

That means `fp_poc` is not a true POC. It is approximately “latest close bucket with cumulative volume snapshot.”

This may be acceptable as a proxy, but it is not production-grade footprint data. If the model was trained on true footprint columns, this is feature drift.

Highest-impact patch order

Fix in this order:



1. Sanitize prices and only exit on valid fresh price.
2. Drop current partial candle from seed/history and prevent restart entries.
3. Move predictor work off the event loop with a predictor lock.
4. Make watchdog heartbeat success-based and track Playwright response tasks.

4. Make watchdog heartbeat success-based and track Playwright response tasks.
5. Add trade sizing guards.
6. Add seeding timeouts.
7. Normalize paths to BASE_DIR / SEED_DIR.
8. Remove hardcoded credentials.
9. Stop returning rawLegends during live polling.
10. Decide whether the Binance footprint proxy is acceptable or replace it with true tick/agg-trade-derived footprint.

The most dangerous single bug is the price=0.0 overwrite path. That one can directly create false exits and corrupt live PnL/trade history.

📄 🍌 💬 ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

🔒 AI Chat

• Genspark vs ChatGPT vs Claude →

Ask anything, create anything

+ | 🌐 GPT-5.5 Pro ▾

⬇️ ↺